Comprehensive Master Test Plan & QA Test Suite

Platform: "Style Haven" – Fashion E-commerce Marketplace

Document Version: 1.0.0-PROD

Author: Principal QA Test Architect

Scope: ALL MODULES (User Accounts, Product Catalog, Shopping Cart, Checkout, Seller Dashboard, Customer Support, Non-Functional, Security)

1. Executive Summary & Strategy

Style Haven connects independent fashion designers with fashion-conscious shoppers aged 18–35. This test suite provides rigorous verification across all architectural tiers:

Client Tier: Web and Mobile Responsive UI, cross-browser consistency, accessibility (WCAG 2.1 AA).

Service & API Tier: RESTful & GraphQL endpoints, authentication tokens (JWT/OAuth), authorization matrices (RBAC/IDOR), idempotency, webhooks.

Data & Concurrency Tier: Distributed inventory locking (Redis/DB row locking), ACID compliance for checkouts, race condition mitigation.

Asynchronous Tier: Email dispatch queues, WebSocket chat sessions, payment gateway callbacks.

QA Resource Assignment Matrix

QA Eng 1: Authentication, User Accounts, Profile, Wishlist, Loyalty, Customer Support (Live Chat/FAQ).

QA Eng 2: Product Catalog, Search & Filtering, Recommendations, Shopping Cart, Inventory Management, Seller Dashboard.

QA Eng 3: Checkout, Payment Gateways (Stripe/PayPal/Wallets), Order Fulfillment, Security, Concurrency, Performance (k6).

2. High-Level Test Scenarios (Matrix Overview)

Module	Positive Scenarios	Negative Scenarios	Boundary & Edge Cases
1. User Accounts	Email/password sign-up, Google/Apple OAuth login, 2FA prompt, profile update, order history view.	Weak password rejection, duplicate email registration, expired JWT token, brute force lockouts.	Max length names/emails, international phone formats, right-to-left (RTL) unicode characters, deleted account login attempt.
2. Product Catalog	Category hierarchy browsing, faceted search (brand, price, size, color), recommendation carousel.	Fuzzy search typos with zero results fallback, corrupted image handling, invalid query parameter injection.	SQL/NoSQL injection via search params, 10,000+ pagination offset, simultaneous variant switching (Size/Color) with out-of-stock permutations.
3. Shopping Cart	Add single/multiple items, quantity increment/decrement, session persistence across tabs, price recalculation.	Adding quantity greater than available stock, adding inactive/delisted products.	Concurrent cart mutation across devices, price updates mid-session, coupon application reducing subtotal to zero or negative.
4. Checkout & Payments	Standard address validation, tiered shipping selection, successful credit card & digital wallet charge, order confirmation email.	Card declined (insufficient funds, CVV mismatch, expired card), webhook network failure.	Payment gateway timeout during 3D Secure verification, double-clicking "Pay Now" button, race condition on 1 remaining stock item across 2 simultaneous buyers.

Module	Positive Scenarios	Negative Scenarios	Boundary & Edge Cases
5. Seller Dashboard	Product listing creation with image upload, real-time inventory increment, order status update (Packed -> Shipped), label generation.	Uploading unsupported file types (>20MB executable), editing orders belonging to other sellers.	IDOR vulnerability on seller inventory modification, bulk CSV import with 5,000 items, low-stock threshold triggers at exact zero.
6. Customer Support	Real-time agent chat initiation, FAQ keyword search, contact form submission with validation.	Submitting empty contact tickets, attachment containing malware payload.	WebSocket reconnect on cellular drop/reconnect, message flood rate-limiting (DDoS prevention), simultaneous multi-tab chat sync.

3. Detailed Step-by-Step Test Cases

Module 1: User Accounts & Authentication

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-ACC-001	Req 1.1	Verify standard user registration with valid credentials	Clean browser session, non-existent email: `shopper.valid@stylehaven.io`	**Given** the user is on the registration page **When** user inputs valid first name, last name, email, and compliant password (12+ chars, symbol, number) **And** clicks "Create Account" **Then** a verification email is dispatched	User account is created with `status: pending_verification`; JWT authentication token is issued; directed to verification prompt.	P1 / Blocker	QA Eng 1
TC-ACC-002	Req 1.1	Verify social OAuth login (Google & Apple)	Google OAuth account configured with valid scopes	**Given** user is on the login modal **When** user clicks "Continue with Google" and authorizes consent screen **Then** session is authenticated	System links Google email, auto-creates customer profile if first-time user, and redirects to intended destination without session desync.	P1 / Blocker	QA Eng 1
TC-ACC-003	Req 1.1	Validate account lockout on repeated failed login attempts (Brute Force Defense)	Registered account `shopper.test@stylehaven.io`	**Given** user attempts login with incorrect password **When** 5 consecutive failed attempts are executed within a 2-minute window **Then** the 6th attempt is blocked with HTTP 429 / Lockout UI	Account is temporarily locked for 15 minutes; security notification email sent to user; generic error prevents email enumeration.	P2 / Critical	QA Eng 1
TC-ACC-004	Req 1.2	Profile management - Update default shipping address & payment details	Authenticated user session	**Given** user is in `Account Settings -> Address Book` **When** user adds a new address with special characters (`Apt #4-B, St. John's Way`) and marks as default **Then** updates are saved to profile	Profile reflects new default address; subsequent checkout loads this address automatically; inputs are sanitized against XSS.	P2 / High	QA Eng 1

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-ACC-005	Req 1.3	Order History pagination and real-time status tracking	User with 25+ completed orders across various states (Delivered, In Transit, Processing)	**Given** user navigates to "My Orders" **When** user views orders and selects page 2, then clicks "Track Package" on an active order **Then** courier tracking modal opens	Orders display accurate dates, SKU thumbnails, itemized costs, and tracking stepper aligns with third-party logistics (3PL) API payload.	P3 / Medium	QA Eng 1

Module 2: Product Catalog, Search & Recommendations

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-CAT-001	Req 2.1	Faceted product search by multiple filters (Category, Size, Price Range, Designer Brand)	Catalog contains 1,000+ populated products	**Given** user is on the Women's Apparel listing page **When** user filters by Size "M", Price "$50-$150", and Brand "Atelier Noir" **Then** filter parameters update in URL and UI updates reactively	Only items matching all active filter criteria are rendered; total product count accurately matches database query results.	P2 / High	QA Eng 2
TC-CAT-002	Req 2.1	Search engine query with typos, special characters, and zero results fallback	Search keyword: `vntage jackett!@#`	**Given** user enters keyword into global search bar **When** user presses Enter **Then** Elasticsearch/Algolia performs fuzzy matching	Search engine suggests "Did you mean: Vintage Jacket?" with relevant matches; zero-result state renders curated "Trending Now" recommendations.	P3 / Medium	QA Eng 2
TC-CAT-003	Req 2.2	Product Detail Page (PDP) variant switching (Size x Color matrix) and inventory reflection	Product SKU with 3 colors (Black, Cream, Olive) and 4 sizes (S, M, L, XL) where Cream-M is out of stock	**Given** user is on PDP for "Linen Oversized Blazer" **When** user clicks "Cream" color swatch, then clicks "M" size button **Then** UI updates price, image gallery, and stock indicator	"Add to Cart" button changes to disabled "Out of Stock / Notify Me"; gallery shows high-res photos for selected color variant only.	P1 / Blocker	QA Eng 2
TC-CAT-004	Req 2.2	Interactive size chart modal and responsive image zoom	Desktop and Mobile viewport	**Given** user is on PDP **When** user hovers over main hero image on desktop or pinches on mobile **And** clicks "Size Guide" **Then** size chart drawer opens	Image zoom magnifies texture without pixelation; size guide provides metric and imperial conversion tables without layout breakage.	P3 / Medium	QA Eng 2

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-CAT-005	Req 2.3	Personalized recommendation algorithm validation	User browsing history contains 5 Bohemian dresses	**Given** authenticated user navigates to Home or PDP **When** recommendation carousel "Recommended For You" loads **Then** items are populated via recommendation engine	Rec engine displays items matching user style preferences, tags, and designer affinity; cold-start users receive top-selling items.	P3 / Medium	QA Eng 2

Module 3: Shopping Cart Management

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-CRT-001	Req 3.1	Add item to cart from PDP with variant parameters	Product with SKU `DRS-SILK-01`, Size: S, Color: Emerald	**Given** user selects Size S and Color Emerald **When** user clicks "Add to Cart" **Then** mini-cart flyout opens	Item is added with correct SKU, price, variant attributes; cart badge counter increments by 1; subtotal updates accurately.	P1 / Blocker	QA Eng 2
TC-CRT-002	Req 3.1	Guest cart migration upon user login	Guest user has 2 items in local storage/cookie cart	**Given** guest user adds items to cart **When** user clicks "Log In" and authenticates successfully **Then** cart state is evaluated	Guest cart items merge with any existing persistent server-side cart for that account; duplicates consolidate quantities up to max inventory.	P2 / Critical	QA Eng 2
TC-CRT-003	Req 3.2	Cart quantity adjustment exceeding stock limit	Item has exactly 3 units left in warehouse	**Given** user has 2 units in cart **When** user clicks `+` to change quantity to 4 **Then** API returns stock availability error	Quantity clamps to 3; user receives warning banner: "Only 3 items available in stock"; subtotal adjusts to 3 units.	P2 / High	QA Eng 2
TC-CRT-004	Req 3.3	Promo code application with subtotal calculation, tax, and free shipping threshold	Cart value: $95. Free shipping threshold: $100. Promo: `SUMMER15` (15% off)	**Given** user enters coupon code `SUMMER15` **When** user clicks "Apply" **Then** discount of $14.25 is deducted	New subtotal = $80.75; shipping fee ($10) is added because discounted subtotal is below $100 threshold; taxes calculated on discounted amount.	P2 / High	QA Eng 2
TC-CRT-005	Req 3.3	Dynamic price update notification while item resides in cart	Designer changes item price from $120 to $100 while user has it in cart	**Given** user left item in cart overnight **When** user re-opens cart page **Then** cart syncs with latest catalog price	Cart recalculates with $100; subtle alert displays: "Price for [Item] changed from $120 to $100"; prevents purchasing at stale price.	P2 / High	QA Eng 2

Module 4: Checkout, Payments & Order Processing

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-CHK-001	Req 4.1	Shipping address validation and carrier rate selection	Valid destination address	**Given** user enters shipping address in checkout step 1 **When** user selects "Express 2-Day Air" ($18.00) **Then** order review summary updates	Total order price immediately reflects shipping surcharge; delivery estimate date displays dynamically based on business days.	P1 / Blocker	QA Eng 3
TC-CHK-002	Req 4.2	Successful end-to-end payment with 3D-Secure Credit Card	Valid Test Visa with 3DS challenge trigger	**Given** user is on Payment step **When** user inputs CC details and clicks "Place Order" **And** completes OTP on 3DS simulator iframe **Then** payment gateway captures charge	Charge succeeds; idempotency key prevents duplicate transaction; webhook confirms payment; user redirected to `/order-success/{orderId}`.	P1 / Blocker	QA Eng 3
TC-CHK-003	Req 4.2	Payment failure handling (Insufficient funds / Card expired)	Test card with declined response code (`insufficient_funds`)	**Given** user submits checkout with declining payment method **When** payment API returns rejection **Then** checkout state remains intact	Clear, non-technical error: "Card was declined. Please try another payment method"; user stays on checkout page with cart contents preserved.	P1 / Blocker	QA Eng 3
TC-CHK-004	Req 4.2	Digital wallet transaction (Apple Pay / Google Pay)	Browser with active Apple Pay / Google Pay session	**Given** user selects "Apple Pay" on checkout **When** biometric authentication (FaceID/TouchID) completes sheet authorization **Then** billing & shipping address sync from wallet	Order creates seamlessly without redundant manual form filling; authorization token validates on backend.	P2 / Critical	QA Eng 3
TC-CHK-005	Req 4.3	Order confirmation trigger & asynchronous email receipt delivery	Successful payment completion	**Given** order is placed successfully **When** message queue (`RabbitMQ` / `Kafka` / `PubSub`) receives `order.created` event **Then** notification service dispatches email	Confirmation email arrives within 60s containing order ID, itemized breakdown, tax invoice, shipping ETA, and cancellation link.	P2 / High	QA Eng 3

Module 5: Seller Dashboard & Order Fulfillment

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-SEL-001	Req 5.1	New product listing creation with media assets & variant pricing	Authenticated seller account	**Given** seller is on `/dashboard/products/new` **When** seller fills title, markdown description, tags, variant SKUs, and uploads 4 WebP images **And** clicks "Publish" **Then** listing becomes active	Product appears in public catalog within 30s cache invalidation; image CDN processes thumbnails; inventory records initialize in DB.	P1 / Blocker	QA Eng 2
TC-SEL-002	Req 5.2	Low stock automated alert notification threshold	Item inventory reaches configured threshold (e.g., <= 2 units)	**Given** a product has 3 units in stock **When** customer purchases 1 unit **Then** remaining count drops to 2	Dashboard badge displays "Low Stock Alert"; automated notification email sent to seller; PDP displays "Only 2 left in stock!".	P2 / High	QA Eng 2

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-SEL-003	Req 5.3	Order fulfillment lifecycle & shipping label PDF generation	Order in `PAID / UNFULFILLED` state assigned to seller	**Given** seller opens order details **When** seller inputs parcel dimensions/weight and clicks "Generate Shipping Label" **Then** tracking number is assigned via carrier API	Printable PDF shipping label opens in browser; order status transitions to `PROCESSING / SHIPPED`; customer tracking link activates.	P1 / Blocker	QA Eng 3
TC-SEL-004	Req 5.3	Prevent seller from viewing or modifying orders belonging to another designer (RBAC)	Seller A (`ID: 101`) and Seller B (`ID: 102`)	**Given** Seller A is authenticated **When** Seller A crafts request to `/api/seller/orders/999` (an order exclusively containing Seller B items) **Then** API denies request	API responds with HTTP 403 Forbidden or 404 Not Found; zero unauthorized order or customer data leaked.	P1 / Blocker	QA Eng 3

Module 6: Customer Support & Additional Features

Test Case ID	Req Ref	Test Scenario & Objective	Pre-conditions & Test Data	Test Steps (Given / When / Then)	Expected Result	Severity / Priority	Assigned Engineer
TC-SUP-001	Req 6.1	Real-time Live Chat session initialization and message exchange	Customer on store front, support agent logged in dashboard	**Given** customer opens chat widget **When** customer sends "I have a question about sizing" **Then** WebSocket establishes connection	Agent receives incoming ticket; bidirectional chat transmits in <500ms; typing indicator and read receipts render correctly.	P2 / High	QA Eng 1
TC-SUP-002	Req 6.2	FAQ accordion interaction and keyword search indexing	FAQ contains 20+ questions across Categories (Returns, Shipping, Payments)	**Given** user navigates to `/faq` **When** user types "refund timeline" into search filter **Then** accordion expands relevant answers	Live filtering isolates relevant entries; clicking accordion toggles smooth CSS open/close animation with proper ARIA attributes.	P3 / Medium	QA Eng 1
TC-OPT-001	Req 7.1	Wishlist toggle, persistence, and move-to-cart action	Authenticated user	**Given** user clicks Heart icon on PDP **When** item is added to Wishlist, and user later clicks "Move to Cart" **Then** item shifts to active cart	Heart icon toggles filled state; item removed from Wishlist and added to Cart with selected size/color preserved.	P3 / Medium	QA Eng 1
TC-OPT-002	Req 7.3	Loyalty points calculation on order placement and checkout redemption	User has 500 Loyalty points ($5 value)	**Given** user reaches checkout payment step **When** user selects "Apply 500 Points for $5 discount" **Then** order balance deducts $5.00	Points balance updates to 0 pending order confirmation; upon order delivery, newly earned points credit to account.	P3 / Medium	QA Eng 1

4. Non-Functional & Security Edge Scenarios

4.1 Race Condition & Inventory Over-Allocation (Double Spend)

Scenario: High-demand flash sale item has exactly **1 unit** remaining in stock. Two customers (User A and User B) simultaneously click "Place Order & Pay" at the exact same millisecond (`T0`).

Test Design:

Execute concurrent JMeter / k6 virtual users sending identical `POST /api/v1/orders/checkout` payloads using Redis distributed locking (`SET key value NX PX 5000`) or database row locking (`SELECT ... FOR UPDATE`).

Expected Result: Exactly **one** order succeeds (HTTP 200/201). The second request fails with HTTP 409 Conflict ("Item was purchased by another shopper"). Warehouse inventory never drops to `-1` .

4.2 Payment Gateway Webhook Loss & Network Recovery

Scenario: Customer completes 3D-Secure authentication on Stripe/PayPal modal, but closes browser tab before redirect back to Style Haven, OR network timeout drops the immediate HTTP response.

Test Design:

Simulate client disconnection immediately upon card capture.

Verify that background webhook listener (`POST /api/v1/webhooks/stripe`) receives `payment_intent.succeeded` .

Expected Result: Backend idempotently fulfills order, assigns order status `PAID` , reserves inventory, and dispatches confirmation email without requiring the user's browser redirect.

4.3 Insecure Direct Object Reference (IDOR) & Privilege Escalation

Scenario: Authenticated standard customer attempts to access `/api/v1/orders/{foreign_order_id}` or seller endpoints `/api/v1/seller/{seller_id}/payouts` .

Test Design:

Automated OWASP ZAP & Postman test suite executing cross-account ID parameter fuzzing.

Expected Result: API returns HTTP 403 Forbidden or 404 Not Found. Zero PII (Personally Identifiable Information) or financial records exposed.

4.4 WebSocket Reconnection & State Recovery (Live Chat)

Scenario: Customer engaged in active Live Chat support loses internet connection (e.g., switches Wi-Fi to cellular or enters tunnel) for 15 seconds.

Test Design:

Simulate `navigator.onLine = false` during WebSocket session, queue 2 outbound messages, then trigger `online` .

Expected Result: Client automatically initiates exponential backoff reconnection, re-authenticates socket with token, replays unsent messages from IndexedDB/memory queue, and reconciles missed incoming messages without conversation loss.

4.5 Cross-Site Scripting (XSS) in Product Reviews & Seller Notes

Scenario: Malicious actor injects or  into Product Review text or Seller Shipping Notes.

Test Design:

Submit dirty payloads through review submission forms and seller portals.

Expected Result: Content is strictly escaped/sanitized on server ingestion and rendered safely as text, preventing arbitrary JavaScript execution in victim browsers.

---

5. Automated Test Feasibility & Framework Recommendations



5.1 Playwright (End-to-End Critical User Journeys)

Playwright is recommended for multi-browser, resilient, parallel E2E testing due to native auto-waiting, network mocking, and headless speed:

Candidate Journey 1: *Guest-to-Paid Customer Flow* (Search -> Filter -> PDP Variant Selection -> Add to Cart -> Guest Checkout -> Card Payment -> Confirmation Screen).

Candidate Journey 2: *Seller Product Lifecycle* (Seller Login -> Create Product Listing -> Validate catalog visibility -> Adjust Stock).

Candidate Journey 3: *Cart State Synchronization* (Mutating cart on Mobile viewport, logging in on Desktop, asserting state consistency).

5.2 Cypress (Frontend Component & Fast Local Dev Feedback)

Candidate Journey 1: PDP interactive components (Size guide drawer, high-res zoom carousel, interactive swatch toggling).

Candidate Journey 2: Client-side Form Validation & Error States (Checkout form empty fields, credit card Luhn check validation, zip code formatting).

5.3 k6 (Performance, Concurrency & Load Testing)

Candidate Test 1 (Flash Sale Concurrency): 500 concurrent virtual users hitting `POST /api/v1/cart/add` and `POST /api/v1/checkout` on 5 limited-edition inventory SKUs to validate locking mechanisms.

Candidate Test 2 (Peak Traffic Stress Test): Simulating 5,000 requests/sec across catalog browse and elasticsearch queries with SLA threshold: `p95 < 350ms`, `error_rate < 0.01%`.

Candidate Test 3 (Sustained Soak Test): 4-hour endurance test at 500 RPS to discover memory leaks in backend microservices and database connection pool exhaustion.

---

6. Test Environment & Entry/Exit Criteria

Entry Criteria for Staging QA

1. All user stories deployed to Staging with green Unit & Integration test pipelines (>85% code coverage).
2. Sanitized seed database populated with minimum 5,000 SKUs, 20 seller accounts, and mock tax/shipping tables.
3. Third-party sandboxes configured and reachable (Stripe Sandbox, SendGrid Sandbox, Shippo/EasyPost Mock).

Exit Criteria for Production Release

1. **Zero Blocker (P1) or Critical (P2)** open defects.
2. All High-Level Test Scenarios (Positive & Negative) successfully executed with artifacts attached.
3. Automated regression suite achieves **100% pass rate** on production-build release candidate.
4. Security sign-off: OWASP Top 10 vulnerabilities verified with zero High/Critical CVEs.
5. Sign-off received from Principal QA Test Architect and Product Owner.